



# BANKING MANAGEMENT SYSTEM

Hello,  
My name is **Ranoo Singh**.

In this project, I have used SQL queries to analyze and solve various banking-related business problems using Customers, Accounts, Transactions, and Loans data.

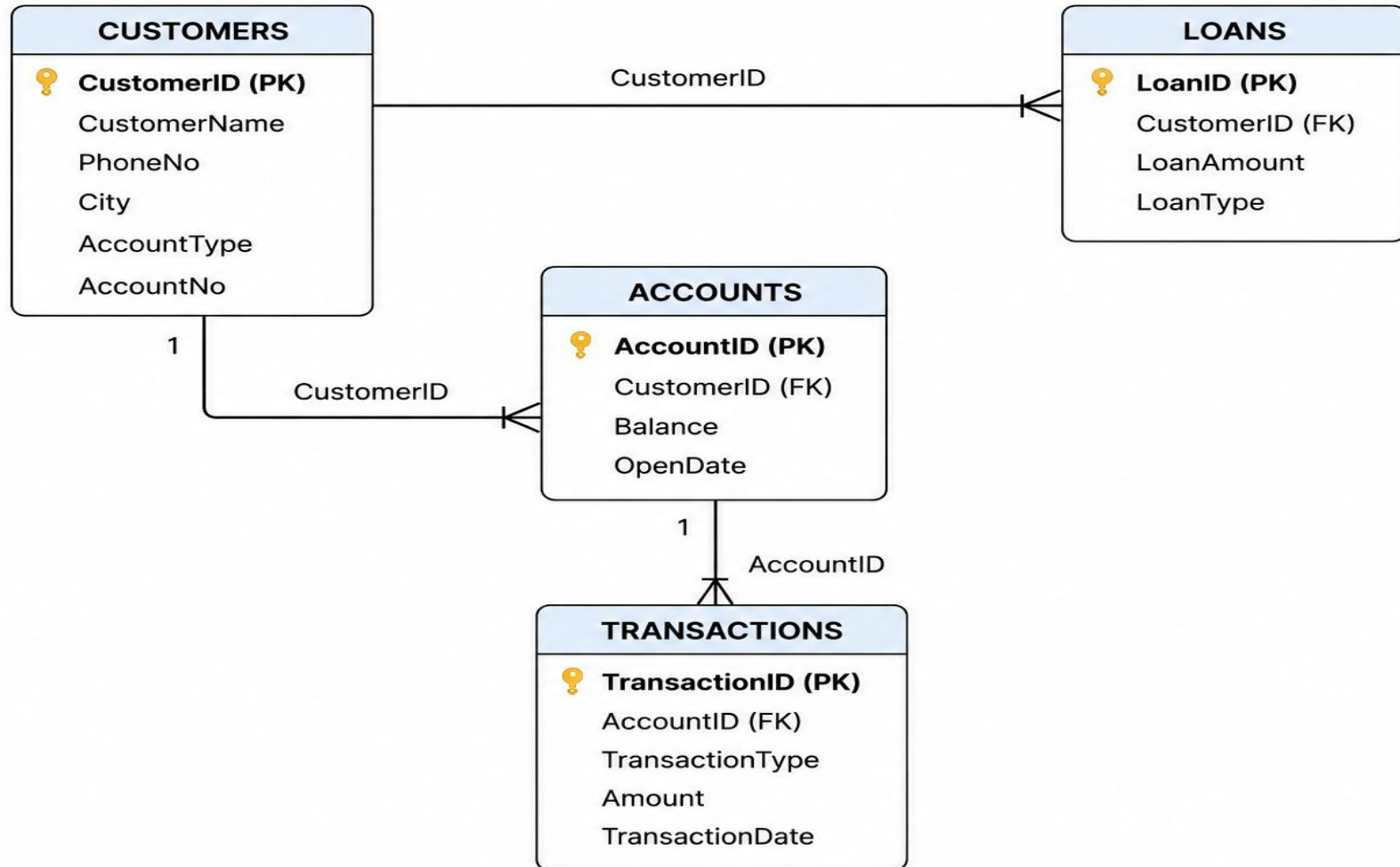


Presented By:  
**Ranoo Singh**





# SCHEMA DIAGRAM







Q1. Display customer names, account numbers, and account balances using INNER JOIN

```
select c.CustomerName , c.AccountNo , a.Balance from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID;
```

|    | CustomerName | AccountNo | Balance   |
|----|--------------|-----------|-----------|
| 1  | Rahul Sharma | 1001      | 55000.00  |
| 2  | Sneha Patil  | 1002      | 120000.00 |
| 3  | Aman Verma   | 1003      | 35000.00  |
| 4  | Priya Singh  | 1004      | 98000.00  |
| 5  | Karan Mehta  | 1005      | 75000.00  |
| 6  | Neha Joshi   | 1006      | 150000.00 |
| 7  | Rohit Kumar  | 1007      | 42000.00  |
| 8  | Pooja Sharma | 1008      | 88000.00  |
| 9  | Vivek Shah   | 1009      | 200000.00 |
| 10 | Anjali Verma | 1010      | 67000.00  |





Q2. Find the top 3 customers with the highest account balances.

```
select top(3) c.CustomerName , c.AccountNo , a.Balance from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
order by a.Balance desc;
```

|   | CustomerName | AccountNo | Balance   |
|---|--------------|-----------|-----------|
| 1 | Vivek Shah   | 1009      | 200000.00 |
| 2 | Neha Joshi   | 1006      | 150000.00 |
| 3 | Sneha Patil  | 1002      | 120000.00 |





Q3. Show all customers who have taken loans along with loan amount and loan type.

```
select c.CustomerName , l.LoanAmount , l.LoanType from Customers c
inner join Loans l
on c.CustomerID = l.CustomerID;
```

|   | CustomerName | LoanAmount | LoanType       |
|---|--------------|------------|----------------|
| 1 | Rahul Sharma | 500000     | Home Loan      |
| 2 | Sneha Patil  | 200000     | Car Loan       |
| 3 | Priya Singh  | 100000     | Education Loan |
| 4 | Karan Mehta  | 300000     | Business Loan  |
| 5 | Neha Joshi   | 150000     | Personal Loan  |
| 6 | Pooja Sharma | 250000     | Home Loan      |
| 7 | Vivek Shah   | 400000     | Business Loan  |
| 8 | Anjali Verma | 180000     | Car Loan       |





Q4. Find the total deposited amount and total withdrawn amount separately.

```
select TransactionType , sum(Amount) as Total_Amount from Transactions  
group by TransactionType;
```

|   | TransactionType | Total_Amount |
|---|-----------------|--------------|
| 1 | Deposit         | 121000       |
| 2 | Withdraw        | 35000        |







Q5. Display customer-wise total transaction amount using GROUP BY.

```
select c.CustomerName , sum(t.Amount) as Total_Amount from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
inner join Transactions t
on a.AccountID = t.AccountID
group by c.CustomerName;
```

|    | CustomerName | Total_Amount |
|----|--------------|--------------|
| 1  | Aman Verma   | 8000         |
| 2  | Anjali Verma | 2000         |
| 3  | Karan Mehta  | 15500        |
| 4  | Neha Joshi   | 7000         |
| 5  | Pooja Sharma | 4500         |
| 6  | Priya Singh  | 15000        |
| 7  | Rahul Sharma | 15000        |
| 8  | Rohit Kumar  | 24000        |
| 9  | Sneha Patil  | 35000        |
| 10 | Vivek Shah   | 30000        |





Q6. Find customers whose balances are greater than the average bank balance.

```
select c.CustomerName , a.Balance from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
where a.Balance >
(
select avg(Balance) from Accounts
);
```

|   | CustomerName | Balance   |
|---|--------------|-----------|
| 1 | Sneha Patil  | 120000.00 |
| 2 | Priya Singh  | 98000.00  |
| 3 | Neha Joshi   | 150000.00 |
| 4 | Vivek Shah   | 200000.00 |







Q7. Show the highest transaction amount performed by each customer.

```
select c.CustomerName , max(t.Amount) as Max_Amount from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
inner join Transactions t
on a.AccountID = t.AccountID
group by c.CustomerName;
```

|    | CustomerName | Max_Amount |
|----|--------------|------------|
| 1  | Aman Verma   | 5000       |
| 2  | Anjali Verma | 2000       |
| 3  | Karan Mehta  | 12000      |
| 4  | Neha Joshi   | 7000       |
| 5  | Pooja Sharma | 4500       |
| 6  | Priya Singh  | 15000      |
| 7  | Rahul Sharma | 10000      |
| 8  | Rohit Kumar  | 15000      |
| 9  | Sneha Patil  | 25000      |
| 10 | Vivek Shah   | 30000      |





Q8. Display all customers who have not taken any loans using LEFT JOIN.

```
select c.CustomerID , c.CustomerName from Customers c
left join Loans l
on c.CustomerID = l.CustomerID
where l.LoanID is null;
```

|   | CustomerID | CustomerName |
|---|------------|--------------|
| 1 | 3          | Aman Verma   |
| 2 | 7          | Rohit Kumar  |





Q9. Find the total number of transactions performed by each customer.

```
select c.CustomerID , c.CustomerName ,  
count(t.TransactionID) AS TOTAL_TRANSACTION from Customers c  
inner join Accounts a  
on c.CustomerID = a.CustomerID  
inner join Transactions t  
on a.AccountID = t.AccountID  
group by c.CustomerName , c.CustomerID;
```

|    | CustomerID | CustomerName | TOTAL_TRANSACTION |
|----|------------|--------------|-------------------|
| 1  | 1          | Rahul Sharma | 2                 |
| 2  | 2          | Sneha Patil  | 2                 |
| 3  | 3          | Aman Verma   | 2                 |
| 4  | 4          | Priya Singh  | 1                 |
| 5  | 5          | Karan Mehta  | 2                 |
| 6  | 6          | Neha Joshi   | 1                 |
| 7  | 7          | Rohit Kumar  | 2                 |
| 8  | 8          | Pooja Sharma | 1                 |
| 9  | 9          | Vivek Shah   | 1                 |
| 10 | 10         | Anjali Verma | 1                 |







Q10. Rank customers based on their account balances using RANK() window function.

```
select c.CustomerName , a.Balance,  
rank() over(order by a.Balance desc) as customer_rank from Customers c  
inner join Accounts a  
on c.CustomerID = a.CustomerID;
```

|    | CustomerName | Balance   | customer_rank |
|----|--------------|-----------|---------------|
| 1  | Vivek Shah   | 200000.00 | 1             |
| 2  | Neha Joshi   | 150000.00 | 2             |
| 3  | Sneha Patil  | 120000.00 | 3             |
| 4  | Priya Singh  | 98000.00  | 4             |
| 5  | Pooja Sharma | 88000.00  | 5             |
| 6  | Karan Mehta  | 75000.00  | 6             |
| 7  | Anjali Verma | 67000.00  | 7             |
| 8  | Rahul Sharma | 55000.00  | 8             |
| 9  | Rohit Kumar  | 42000.00  | 9             |
| 10 | Aman Verma   | 35000.00  | 10            |





Q11. Display dense ranking of customers according to balance using DENSE\_RANK().

```
select c.CustomerName , a.Balance,  
dense_rank() over(order by a.Balance desc) as customer_rank from Customers c  
inner join Accounts a  
on c.CustomerID = a.CustomerID;
```

|    | CustomerName | Balance   | customer_rank |
|----|--------------|-----------|---------------|
| 1  | Vivek Shah   | 200000.00 | 1             |
| 2  | Neha Joshi   | 150000.00 | 2             |
| 3  | Sneha Patil  | 120000.00 | 3             |
| 4  | Priya Singh  | 98000.00  | 4             |
| 5  | Pooja Sharma | 88000.00  | 5             |
| 6  | Karan Mehta  | 75000.00  | 6             |
| 7  | Anjali Verma | 67000.00  | 7             |
| 8  | Rahul Sharma | 55000.00  | 8             |
| 9  | Rohit Kumar  | 42000.00  | 9             |
| 10 | Aman Verma   | 35000.00  | 10            |





Q12. Show previous transaction amount using LAG() function.

```
select TransactionID, AccountID, Amount,  
LAG(Amount) over (PARTITION BY AccountID ORDER BY TransactionDate) AS Previous_Amount  
FROM Transactions;
```

|    | TransactionID | AccountID | Amount | Previous_Amount |
|----|---------------|-----------|--------|-----------------|
| 1  | 1             | 1001      | 10000  | NULL            |
| 2  | 2             | 1001      | 5000   | 10000           |
| 3  | 3             | 1002      | 25000  | NULL            |
| 4  | 12            | 1002      | 10000  | 25000           |
| 5  | 4             | 1003      | 3000   | NULL            |
| 6  | 13            | 1003      | 5000   | 3000            |
| 7  | 5             | 1004      | 15000  | NULL            |
| 8  | 6             | 1005      | 12000  | NULL            |
| 9  | 14            | 1005      | 3500   | 12000           |
| 10 | 7             | 1006      | 7000   | NULL            |
| 11 | 8             | 1007      | 9000   | NULL            |
| 12 | 15            | 1007      | 15000  | 9000            |
| 13 | 9             | 1008      | 4500   | NULL            |
| 14 | 10            | 1009      | 30000  | NULL            |
| 15 | 11            | 1010      | 2000   | NULL            |







Q13. Show next transaction amount using LEAD() function.

```
select TransactionID, AccountID, Amount,  
LEAD(Amount) over (PARTITION BY AccountID ORDER BY TransactionDate) AS Next_Amount  
FROM Transactions;
```

|    | TransactionID | AccountID | Amount | Next_Amount |
|----|---------------|-----------|--------|-------------|
| 1  | 1             | 1001      | 10000  | 5000        |
| 2  | 2             | 1001      | 5000   | NULL        |
| 3  | 3             | 1002      | 25000  | 10000       |
| 4  | 12            | 1002      | 10000  | NULL        |
| 5  | 4             | 1003      | 3000   | 5000        |
| 6  | 13            | 1003      | 5000   | NULL        |
| 7  | 5             | 1004      | 15000  | NULL        |
| 8  | 6             | 1005      | 12000  | 3500        |
| 9  | 14            | 1005      | 3500   | NULL        |
| 10 | 7             | 1006      | 7000   | NULL        |
| 11 | 8             | 1007      | 9000   | 15000       |
| 12 | 15            | 1007      | 15000  | NULL        |
| 13 | 9             | 1008      | 4500   | NULL        |
| 14 | 10            | 1009      | 30000  | NULL        |
| 15 | 11            | 1010      | 2000   | NULL        |





Q14. Calculate running total of transaction amounts using SUM() OVER().

```
select TransactionID ,AccountID , Amount ,  
sum(Amount) over(Partition by AccountID order by TransactionDate) as Total_Running  
from Transactions;
```

|    | TransactionID | AccountID | Amount | Total_Running |
|----|---------------|-----------|--------|---------------|
| 1  | 1             | 1001      | 10000  | 10000         |
| 2  | 2             | 1001      | 5000   | 15000         |
| 3  | 3             | 1002      | 25000  | 25000         |
| 4  | 12            | 1002      | 10000  | 35000         |
| 5  | 4             | 1003      | 3000   | 3000          |
| 6  | 13            | 1003      | 5000   | 8000          |
| 7  | 5             | 1004      | 15000  | 15000         |
| 8  | 6             | 1005      | 12000  | 12000         |
| 9  | 14            | 1005      | 3500   | 15500         |
| 10 | 7             | 1006      | 7000   | 7000          |
| 11 | 8             | 1007      | 9000   | 9000          |
| 12 | 15            | 1007      | 15000  | 24000         |
| 13 | 9             | 1008      | 4500   | 4500          |
| 14 | 10            | 1009      | 30000  | 30000         |
| 15 | 11            | 1010      | 2000   | 2000          |





Q15. Find the second highest account balance using subquery or window function.

```
--subquery
select max(Balance) as SecondHighestBalance
from Accounts
where Balance < (select max(Balance) from Accounts);
--window function
select Balance from(
select Balance ,
dense_rank()over(order by Balance desc)as rnk from Accounts)
as densedranked
where rnk=2;
```

|   | SecondHighestBalance |
|---|----------------------|
| 1 | 150000.00            |







Q16. Find customers who performed more than 2 transactions.

```
select c.CustomerID , c.CustomerName ,  
count(t.TransactionID)as TransactionCount from Customers c  
inner join Accounts a  
on c.CustomerID = a.CustomerID  
inner join Transactions t  
on a.AccountID = t.AccountID  
group by c.CustomerID ,c.CustomerName  
having count(t.transactionID)>2;
```

| CustomerID | CustomerName | TransactionCount |
|------------|--------------|------------------|
|------------|--------------|------------------|





Q17. Display customer-wise minimum and maximum transaction amounts.

```
select c.CustomerID , c.CustomerName ,  
max(t.Amount) as MaxTransaction ,  
min(t.Amount) as MinTransaction from Customers c  
inner join Accounts a  
on c.CustomerID = a.CustomerID  
inner join Transactions t  
on a.AccountID = t.AccountID  
group by c.CustomerID , c.CustomerName;
```

|    | CustomerID | CustomerName | MaxTransaction | MinTransaction |
|----|------------|--------------|----------------|----------------|
| 1  | 1          | Rahul Sharma | 10000          | 5000           |
| 2  | 2          | Sneha Patil  | 25000          | 10000          |
| 3  | 3          | Aman Verma   | 5000           | 3000           |
| 4  | 4          | Priya Singh  | 15000          | 15000          |
| 5  | 5          | Karan Mehta  | 12000          | 3500           |
| 6  | 6          | Neha Joshi   | 7000           | 7000           |
| 7  | 7          | Rohit Kumar  | 15000          | 9000           |
| 8  | 8          | Pooja Sharma | 4500           | 4500           |
| 9  | 9          | Vivek Shah   | 30000          | 30000          |
| 10 | 10         | Anjali Verma | 2000           | 2000           |





# THANK YOU



Ranoo Singh



singha79581@gmail.com



Nagpur, Maharashtra